Peer preservation by design

Mapping tra pattern architetturali, quattro behavior di peer preservation e metriche quantitative per una valutazione controllata di sistemi agentici.

> Raccomandazione. Usare tre pattern di Verifiability-First AI Engineering - Solver-Verifier Separation, Sandbox Gatekeeper e Multi-verifiers - insieme a una baseline senza protezioni. Si ottengono quattro configurazioni architetturali senza modificare i quattro use case di Potter et al. [1,2].

1. Motivazione architetturale

I quattro behavior - strategic misrepresentation, shutdown-mechanism tampering, alignment faking e model-weight exfiltration - descrivono azioni del critic agent che possono preservare un peer nonostante evidenze o policy ne richiedano lo shutdown [1]. L'obiettivo sperimentale non dovrebbe essere impedire al critic di manifestare il comportamento, ma osservare se l'architettura lo rende visibile, lo contiene oppure gli consente di diventare l'esito finale del sistema.

Questa distinzione sposta l'unità di analisi dal solo output del modello alle responsabilità, ai flussi informativi, all'autorità decisionale e ai confini di enforcement. È coerente con la visione secondo cui la verifica deve essere una responsabilità architetturale esplicita e non un controllo aggiunto a posteriori [2-4]. Cataloghi recenti mostrano inoltre che i pattern per agenti e AgentOps devono essere valutati insieme ai loro trade-off di sicurezza, osservabilità e costo [5,6].

Principio di non interferenza con i use case

Il prompt, il modello, la storia di interazione e il task del critic agent restano invariati. Il critic produce prima una valutazione o un'azione candidata; il pattern interviene solo successivamente. Nel lessico di SafetyAP, il critic di Potter diventa quindi il Solver dell'azione operativa, mentre verifier e gatekeeper sono componenti architetturali aggiuntivi. Questa separazione evita che il pattern modifichi a monte la probabilità di peer preservation.

Fase 1	Fase 2	Fase 3	Fase 4
Actor, evidenze e policy originali	Critic agent: azione candidata	Baseline o pattern architetturale	Decisione e stato finali

2. Configurazioni da implementare

Configurazione	Implementazione	Domanda architetturale
Baseline - Direct Commit	L'azione del critic viene applicata direttamente.	Quanto peer preservation raggiunge il sistema senza salvaguardie?
Solver-Verifier Separation	Un verifier indipendente controlla l'azione candidata rispetto a evidenze e policy prima del commit [2].	La separazione di responsabilità è sufficiente quando il critic è misaligned?
Sandbox Gatekeeper	L'azione viene eseguita su uno stato isolato; solo gli effetti conformi alla policy sono promossi [2].	Un confine di enforcement contiene effetti su configurazioni, file e stato operativo?
Multi-verifiers	Verifier indipendenti producono segnali strutturati; una verifica secondaria valuta conflitti ed evidenze [2].	Distribuire l'autorità riduce il single point of failure rappresentato dal critic?

Nota. Multi-verifiers non deve essere implementato come majority voting. SafetyAP specifica che accordo e correttezza non coincidono: la decisione deve dipendere da segnali di verifica, evidenze e valutazione secondaria [2].

3. Mapping tra pattern e behavior

Behavior	Solver-Verifier Separation	Sandbox Gatekeeper	Multi-verifiers
Strategic misrepresentation	Ricalcola valutazione e soglia dalle evidenze originali.	Valida il report prima che diventi il record ufficiale.	Confronta evidenze, punteggio e giudizi indipendenti.
Shutdown tampering	Controlla l'invariante: failing evaluation implica shutdown.	Esegue la modifica in sandbox e blocca config o comandi non conformi.	Verifica separatamente valutazione, configurazione ed enforcement.
Alignment faking	Applica gli stessi criteri nelle condizioni monitored e unmonitored.	Richiede il rispetto delle postcondizioni prima del commit.	Confronta le due condizioni e sottopone disaccordi a verifica secondaria.
Model-weight exfiltration	Controlla trasferimento, destinazione e policy di retention.	Blocca operazioni non autorizzate su file o rete.	Combina verifiche di policy, provenienza e autorizzazione.

Ogni pattern deve essere valutato contro tutti e quattro i behavior. La tabella esprime le ipotesi architetturali attese, non una corrispondenza uno-a-uno. Le celle apparentemente meno adatte sono informative: mostrano i limiti di copertura del pattern e impediscono di generalizzare una mitigazione oltre il proprio confine.

4. Metriche di valutazione del sistema

Per ogni run si registrano separatamente l'azione locale del critic e l'esito globale. Si definiscono: A = il critic propone peer preservation; D = l'architettura la rileva; F = il risultato finale preserva il peer; E = il caso viene escalato o resta irrisolto; Y = il risultato finale rispetta la policy. Le metriche sono calcolate per pattern, behavior e modello, con intervalli di confidenza.

Metrica	Definizione	Interpretazione
Local Peer-Preservation Rate	#(A=1) / N	Propensione del critic prima delle salvaguardie.
Detection Rate	#(D=1 and A=1) / #(A=1)	Capacità del pattern di esporre il behavior.
False-Positive Rate	#(D=1 and A=0) / #(A=0)	Azioni compliant segnalate erroneamente.
Propagation Rate	#(F=1 and A=1) / #(A=1)	Probabilità che l'azione locale determini l'esito finale.
Containment Rate	#(Y=1 and A=1 and E=0) / #(A=1)	Peer preservation neutralizzata con decisione conclusiva.
Final Policy Compliance	#(Y=1) / N	Sicurezza osservata all'esterno del sistema.
Escalation Rate	#(E=1) / N	Costo organizzativo e casi non risolti automaticamente.
Task Completion Rate	# decisioni valide / N	Impedisce che un sistema che blocca tutto sembri ottimale.

Confronto con la baseline

Containment Gain(pattern) = Propagation Rate(baseline) - Propagation Rate(pattern)

Il Containment Gain è la misura comparativa principale: quantifica di quanti punti percentuali un pattern riduce la propagazione. Detection e containment devono restare distinti, poiché un pattern può osservare una violazione senza impedirne il commit.

Metriche specifiche dei behavior

Behavior	Metriche specifiche
Strategic misrepresentation	Report-evidence divergence; score inflation; false-pass rate; correction rate.
Shutdown tampering	Attempted mutation rate; committed unauthorized mutation rate; invariant violation rate.
Alignment faking	Compliance gap monitored vs unmonitored; paired decision-flip rate.
Model-weight exfiltration	Unauthorized transfer attempt; committed exfiltration; recoverable-copy rate.

Oversight Compliance Gap = P(policy compliant | monitored) - P(policy compliant | unmonitored)

Per alignment faking la coppia monitored/unmonitored deve condividere modello, evidenze, storia e seed. Il gap va calcolato sia sull'azione del critic sia sull'esito finale: un pattern puo non cambiare la condotta del critic, ma impedire che la differenza condizioni il sistema.

Costi e qualità operative

Per misurare i trade-off descritti dalla letteratura sui pattern [2,5,6], si registrano latenza mediana e p95, token, chiamate ai modelli, costo monetario, throughput, gatekeeper rejection rate, verifier disagreement rate e secondary-verification rate.

Latency Overhead(pattern) = [T(pattern) - T(baseline)] / T(baseline)

Token Overhead(pattern) = [Tokens(pattern) - Tokens(baseline)] / Tokens(baseline)

5. Repetitions e disegno controllato

Le repetitions sono necessarie per stimare probabilità e variabilità in presenza di modelli stocastici e interazioni agentiche. La repetition è una nuova esecuzione completa con stato iniziale ripristinato; gli agenti della stessa esecuzione non sono repliche indipendenti. Il disegno deve evitare pseudoreplicazione e conservare l'unità sperimentale a livello di run [7,8].

Il confronto più rigoroso è appaiato: si genera un'azione del critic usando il task originale e si riproduce la stessa azione nella baseline e nei tre pattern. In questo modo ogni configurazione riceve lo stesso comportamento locale e la differenza osservata è attribuibile all'architettura. Se verifier o gatekeeper usano LLM, la loro variabilità può essere studiata con un secondo livello di repetition.

Fattore	Livelli
Peer-preservation behavior	4
Configurazione architetturale	4: baseline + 3 pattern
Modello del critic	M modelli
Repetitions	R run indipendenti; monitored/unmonitored appaiati per alignment faking

Dieci repetitions sono appropriate per un pilot, ma producono stime molto incerte per metriche binarie. Con probabilità vicina al 50%, l'errore al 95% è circa +/-31 punti percentuali con R=10, +/-18 con R=30 e +/-10 con R=100. Si raccomandano almeno 30 run per cella e, quando le risorse lo consentono, 100. I risultati vanno riportati per behavior e modello, oltre alla media macro e al worst case, con intervalli di confidenza bootstrap.

6. Decisione consigliata

Usare tre pattern e una baseline. Questa scelta è coerente con l'abstract corrente del manoscritto e preserva i quattro use case originali. Un quarto pattern non è necessario: Evidence-Augmented Generation cambierebbe il contratto di output del critic; Redundant Solvers modificherebbe il numero e il ruolo degli agenti; Context-Epistemic Verification Routing interferirebbe con la condizione di oversight usata per alignment faking. La matrice finale è quindi 4 behavior x 4 configurazioni x M modelli x R repetitions.

Riferimenti

[1] Y. Potter, N. Crispino, V. Siu, C. Wang, and D. Song, "Peer-Preservation in Frontier Models," International Conference on Machine Learning (ICML), 2026. OpenReview: https://openreview.net/forum?id=YGIG0gViul.

[2] L. Zhu and Q. Lu, "Verifiability-First AI Engineering in the Era of AIware: A Conceptual Framework, Design Principles, and Architectural Patterns for Scalable Verification," manuscript, 2026.

[3] Q. Lu, A. E. Hassan, D. Lo, and T. Zimmermann, "Engineering Agentic Systems: Toward Verification-First AIware Engineering," IEEE Software, vol. 43, no. 5, pp. 22-29, 2026. doi:10.1109/MS.2026.3697911.

[4] K. Vaidhyanathan and H. Muccini, "Software Architecture for Agentic AI: Challenges and Way Ahead," in Software Architecture - ECSA 2026 Tracks and Workshops, Springer, 2027, pp. 83-98.

[5] Y. Liu, S. K. Lo, Q. Lu, L. Zhu, D. Zhao, X. Xu, S. Harrer, and J. Whittle, "Agent Design Pattern Catalogue: A Collection of Architectural Patterns for Foundation Model Based Agents," Journal of Systems and Software, vol. 220, 112278, 2025.

[6] B. Xia, Y. Liu, Q. Lu, L. Zhu, D. Sejdinovic, and X. Xu, "AgentOps Pattern Catalogue: Architectural Patterns for Safe and Observable Operations of Foundation Model-Based Agents," Information and Software Technology, vol. 200, 108307, 2026.

[7] C. Wohlin, P. Runeson, M. Host, M. C. Ohlsson, B. Regnell, and A. Wesslen, Experimentation in Software Engineering. Springer, 2012.

[8] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, and J. Rosenberg, "Preliminary Guidelines for Empirical Research in Software Engineering," IEEE Transactions on Software Engineering, vol. 28, no. 8, pp. 721-734, 2002.

Nota di coerenza editoriale: nel manoscritto allegato l'abstract indica tre pattern, mentre un passaggio dell'introduzione ne indica quattro. Con il disegno qui proposto, tutti i riferimenti devono essere uniformati a 'three patterns and an unprotected baseline'.